

ALIKA-KONNECT

Connecter. Créer. Inspirer.

CAHIER DES CHARGES FONCTIONNEL & TECHNIQUE

LYNE RESTAURANT

Système interne de gestion de restaurant

Élément	Valeur
Client	LYNE RESTAURANT
Prestataire	ALIKA-KONNECT
Type	Application interne de gestion / POS / Stock / Personnel / Finance
Méthode	Développement incrémental, validation phase par phase
Version du document	1.0 — 23 septembre 2026
Budget de référence	

DOCUMENT MAÎTRE POUR DÉVELOPPEURS ET AGENTS D'INTELLIGENCE ARTIFICIELLE

0. Mode d'emploi de ce document

Ce cahier des charges est le document maître du projet. Il doit pouvoir être transmis à un développeur humain ou à un agent d'intelligence artificielle sans dépendre d'explications orales supplémentaires. Toute implémentation doit respecter les exigences, rôles, règles métier, phases et critères d'acceptation ci-dessous.

- Ne pas développer toutes les phases en une seule fois.
- Une phase doit être développée, testée, documentée et validée avant de commencer la suivante.
- Ne pas supprimer ni réécrire une fonctionnalité déjà validée sans instruction explicite.
- À la fin de chaque phase : fournir les fichiers modifiés, migrations SQL, tests effectués, résultats, limites et instructions de déploiement.
- Toute hypothèse non définie dans ce document doit être signalée avant implémentation lorsqu'elle modifie le métier, les coûts ou les données.
- Le code doit être versionné avec Git ; un commit clair doit clôturer chaque phase validée.
- Les données financières, stocks et journaux d'audit ne doivent jamais être supprimés silencieusement.

1. Contexte et vision

LYNE RESTAURANT souhaite numériser ses opérations internes. La solution n'est pas une plateforme de commande à distance : les clients commandent physiquement au restaurant. L'application doit centraliser le personnel, le menu, les commandes, la caisse, les tickets, les achats, les stocks, les dépenses, les ventes, les rapports et la traçabilité.

1.1 Objectifs principaux

- Réduire les erreurs de prise de commande et de calcul.
- Suivre une commande de sa création jusqu'à sa clôture.
- Imprimer des tickets/factures thermiques brandés LYNE RESTAURANT.
- Connaître les ventes, recettes, dépenses et mouvements de stock par période.
- Donner au propriétaire une vision globale et au personnel uniquement les accès nécessaires.
- Conserver un historique fiable des opérations et des utilisateurs responsables.
- Permettre la gestion des employés et la génération de cartes de service.
- Préparer une architecture évolutive sans inclure de commande client à distance dans la V1.

1.2 Hors périmètre V1

- Commande en ligne par le client depuis Internet.
- Application mobile publique pour les clients.
- Livraison à domicile et suivi GPS des livreurs.
- Marketplace multi-restaurants.
- Comptabilité générale complète au sens d'un logiciel comptable certifié.
- Paie complète, déclarations fiscales et gestion RH avancée.
- Paiement automatique via API Mobile Money, sauf ajout futur validé.

2. Utilisateurs, rôles et permissions

Rôle	Mission	Accès essentiel
Administrateur / Propriétaire	Pilotage global	Tous les modules, utilisateurs, paramètres, rapports, finances, stocks, audit
Gérant	Gestion opérationnelle	Achats, fournisseurs, réception stock, inventaire, consultation ventes/commandes selon permission
Réceptionniste / Caissière	Commande et encaissement	Menu disponible, commandes, paiement, ticket, historique limité
Cuisine (option écran)	Préparation	Commandes à préparer, changement En

Le système doit utiliser un contrôle d'accès par rôles et permissions. Les menus, routes API et actions sensibles doivent être protégés côté serveur ; masquer un bouton dans l'interface ne constitue pas une sécurité suffisante.

3. Parcours métier principal

3.1 Vente / commande

1. La réceptionniste ouvre une nouvelle commande.
2. Elle choisit Table / Comptoir et, si nécessaire, saisit le numéro de table.
3. Elle ajoute les plats/produits et quantités ; le système calcule les montants.
4. La commande reçoit un numéro unique et l'état « Nouvelle ».
5. La cuisine peut la voir ; elle passe à « En préparation », puis « Prête ».
6. La réceptionniste confirme « Servie ».
7. Le paiement est enregistré avec son moyen de paiement.
8. La commande devient « Payée / Clôturée ».
9. Le ticket thermique est généré et peut être imprimé.
10. La vente, la caisse et les statistiques sont actualisées.

3.2 Achat / approvisionnement

11. Le gérant crée une facture d'achat et sélectionne/crée le fournisseur.
12. Il ajoute les articles, quantités, unités, prix unitaires et justificatif éventuel.
13. Le total est calculé automatiquement.
14. La réception de l'achat génère les entrées de stock correspondantes.
15. La dépense liée est enregistrée dans les finances.
16. L'opération apparaît dans le journal d'audit et dans les rapports.

4. Modules fonctionnels

4.1 Authentification et sécurité

- Connexion par identifiant/e-mail et mot de passe ; déconnexion ; session sécurisée.
- Réinitialisation de mot de passe par administrateur au minimum.
- Blocage/désactivation d'un compte sans supprimer son historique.
- Journalisation des connexions et actions sensibles.
- Limitation des tentatives de connexion et validation serveur de toutes les entrées.

4.2 Tableau de bord Administrateur

- Indicateurs : ventes du jour, commandes, recettes, dépenses, solde de caisse, achats, alertes stock.
- Graphiques par jour/semaine/mois/année et période personnalisée.
- Produits/plats les plus vendus et état des commandes.
- Accès rapide aux alertes, rapports et opérations récentes.

4.3 Personnel

- Fiche employé : matricule, photo, nom, sexe, téléphone, adresse, fonction, service, date d'engagement, statut, salaire facultatif, observations.
- Fonctions et services configurables.
- Association facultative d'un employé à un compte utilisateur.

- Génération d'une carte de service imprimable avec logo, photo, nom, matricule, fonction et QR Code optionnel.

4.4 Catalogue / Menu

- Catégories de vente : plats, boissons, accompagnements, etc.
- Produit vendu : nom, code, catégorie, description, prix, photo, disponibilité, statut.
- Historique ou audit des changements de prix.
- Un produit indisponible ne doit pas être ajoutable à une nouvelle commande.

4.5 Commandes

- Création, modification avant clôture, consultation et recherche.
- Numéro unique automatique de type CMD-AAAA-XXXX.
- États : Nouvelle, En préparation, Prête, Servie, Payée/Clôturée, Annulée.
- Annulation protégée avec motif obligatoire et permission appropriée.
- Calcul serveur du sous-total et du total ; ne jamais faire confiance au total envoyé par le client.
- Horodatage des changements d'état et identification de l'utilisateur.

4.6 Caisse, paiements et tickets

- Paiement lié à une commande ; modes : Espèces, Mobile Money, Banque/Carte, Autre.
- Numéro de ticket/facture unique.
- Ticket 58 mm et/ou 80 mm configurable : logo, restaurant, adresse, téléphone, numéro, date/heure, lignes, total, mode de paiement, caissier et message.
- Réimpression autorisée et journalisée ; un ticket réimprimé ne crée pas une seconde vente.
- Gestion d'une session de caisse recommandée : ouverture, encaissements, sorties autorisées, clôture, écart.

4.7 Stocks

- Articles de stock distincts du menu : ingrédients, boissons, consommables et emballages.
- Champs : code, nom, catégorie, unité, quantité disponible, seuil d'alerte, coût indicatif, actif/inactif.
- Mouvements : entrée, sortie, ajustement, perte/casse, inventaire.
- Historique immuable des mouvements avec quantité avant/après, motif, référence et utilisateur.
- Alertes Stock faible et Rupture.

4.8 Achats et fournisseurs

- Fournisseurs : nom, contact, téléphone, adresse, notes.
- Factures d'achat : référence, date, fournisseur, lignes, quantités, unités, prix, total, mode de paiement, justificatif, statut.
- Validation/réception d'un achat : entrée de stock + dépense associée sans double saisie.
- Une facture validée ne doit pas être supprimée ; correction par annulation/contre-opération autorisée et audité.

4.9 Finances

- Recettes issues des ventes et autres recettes autorisées.
- Dépenses : achats, transport, entretien, gaz, salaires ou autres catégories configurables.
- Suivi par période et par mode de paiement.
- Distinguer chiffre d'affaires, recettes encaissées, dépenses et solde ; ne pas présenter automatiquement recettes - dépenses comme bénéfice comptable.

4.10 Rapports

- Rapports journalier, hebdomadaire, mensuel, annuel et période personnalisée.
- Ventes, commandes, paiements, achats, dépenses, stocks, mouvements, produits les plus vendus, activité utilisateurs.
- Exports PDF/Excel et impression lorsque techniquement applicable.

4.11 Notifications

- Alertes internes pour stock faible, rupture, opérations nécessitant attention et erreurs métier pertinentes.
- JavaScript/TypeScript gère l’affichage des alertes UI ; les règles et autorisations critiques restent validées côté API.

4.12 Journal d’audit

- Enregistrer utilisateur, action, module, objet, identifiant, date/heure, anciennes/nouvelles valeurs lorsque pertinent et adresse IP si disponible.
- Actions sensibles : connexion, création/modification utilisateur, prix, annulation commande, achat, dépense, ajustement stock, réimpression, paramètres.

4.13 Paramètres

- Identité LYNE RESTAURANT : logo, nom, adresse, téléphones, devise, message ticket.
- Configuration imprimante/taille ticket.
- Gestion des catégories, unités, moyens de paiement, rôles et permissions.
- Paramètres de numérotation.

5. Règles métier essentielles

ID	Règle
RG-01	Une commande clôturée ne peut plus être modifiée directement.
RG-02	Toute annulation doit avoir un motif et un utilisateur autorisé.
RG-03	Le total d’une commande est recalculé côté serveur à partir des prix applicables.
RG-04	Un paiement ne doit jamais être comptabilisé deux fois.
RG-05	Une réimpression de ticket ne génère ni vente ni paiement supplémentaire.
RG-06	Une réception d’achat validée crée les mouvements d’entrée de stock une seule fois.
RG-07	Les quantités de stock ne doivent pas être modifiées directement : utiliser un mouvement ou un inventaire.
RG-08	Les suppressions physiques des données financières, achats, ventes et mouvements sont interdites en exploitation normale.
RG-09	Chaque opération sensible conserve son auteur et son horodatage.
RG-10	Le système doit gérer USD et permettre de configurer la devise d’affichage ; toute multi-devise réelle nécessite une règle de taux séparée.
RG-11	Le statut d’un employé et celui de son compte utilisateur sont gérés séparément.
RG-12	Le propriétaire peut consulter l’ensemble des rapports ; les autres rôles sont limités par permission.

6. Architecture technique cible

Architecture recommandée : monorepo séparant application mobile/tablette, application web d’administration si retenue, API et base de données. React Native est utilisé pour les applications opérationnelles. JavaScript/TypeScript est utilisé pour la logique client et les alertes. MySQL est la source de vérité des données.

Couche	Technologie cible	Usage
Applications	React Native + Expo (TypeScript recommandé)	Android en priorité ; interfaces Réception/Gérant et éventuellement Admin
Web Admin (recommandé)	React + Vite + TypeScript	Dashboard propriétaire sur ordinateur/tablette
UI mobile	React Native Paper ou composants internes cohérents	Composants, formulaires, navigation
Navigation	Expo Router ou React Navigation	Routes et écrans
État / données serveur	TanStack Query + état local léger	Cache, requêtes, synchronisation UI
Alertes UI	JavaScript/TypeScript + composants	Messages succès, avertissement, erreur

	Toast/Dialog	
API	Node.js LTS + Express ou Fastify, TypeScript recommandé	REST API, logique métier, RBAC, impression/exports
Base de données	MySQL 8+	Persistence relationnelle
Accès DB	Prisma ORM ou requêtes SQL paramétrées	Migrations et accès sécurisé
Authentification	Sessions/token sécurisé + hash Argon2/bcrypt	Connexion et autorisations
Validation	Zod/Joi côté API	Validation stricte des payloads
Fichiers	Stockage serveur objet/fichiers + références DB	Photos, logos, justificatifs
Impression	ESC/POS réseau/Bluetooth/USB selon matériel final	Tickets thermiques 58/80 mm
Exports	PDF + XLSX côté serveur	Rapports
Versionnement	Git + dépôt privé	Historique et phases
Déploiement	HTTPS, variables d'environnement, sauvegardes MySQL	Production

Important : le modèle exact de mini-imprimante doit être confirmé avant de figer le pilote d'impression. Bluetooth, USB et réseau n'utilisent pas toujours la même intégration, particulièrement sur Android/iOS.

6.1 Organisation de dépôt recommandée

```
lyne-restaurant/
├── apps/
│   ├── mobile/           # React Native / Expo
│   └── admin-web/        # React/Vite (si retenu)
├── services/
│   └── api/              # Node.js API
├── packages/
│   ├── shared/          # types, validation, constantes
│   └── ui/              # composants partagés si pertinent
├── database/
│   ├── migrations/
│   └── seed/
├── docs/
│   ├── cahier-des-charges/
│   └── api/
├── .env.example
└── README.md
```

7. Modèle de données minimum

Table	Finalité
users	Comptes et rôles
roles	Rôles
permissions	Permissions
role_permissions	Association rôles/permissions
employees	Personnel
departments	Services
job_titles	Fonctions
menu_categories	Catégories du menu
menu_items	Plats/produits vendus
menu_price_history	Historique prix
orders	Commandes
order_items	Lignes de commande
order_status_history	Historique des états
payments	Paielements
cash_sessions	Sessions de caisse
stock_categories	Catégories stock
stock_items	Articles stock
stock_movements	Mouvements stock
stock_counts	Inventaires
suppliers	Fournisseurs

purchases	Achats
purchase_items	Lignes d'achat
expenses	Dépenses
expense_categories	Catégories dépenses
notifications	Notifications
audit_logs	Journal d'audit
settings	Paramètres
attachments	Fichiers/justificatifs

Toutes les tables métier doivent comporter un identifiant stable, created_at, updated_at lorsque pertinent. Les montants doivent utiliser DECIMAL et jamais FLOAT. Les clés étrangères, index et contraintes d'unicité doivent être définis.

8. API REST – domaines minimums

- /auth : login, logout, session/me, changement de mot de passe.
- /users, /roles, /permissions, /employees.
- /menu/categories, /menu/items.
- /orders et /orders/{id}/status.
- /payments et /cash-sessions.
- /stock/items, /stock/movements, /stock/counts.
- /suppliers, /purchases et réception/validation.
- /expenses et /finance/summary.
- /reports.
- /notifications.
- /audit-logs.
- /settings et /files.

Les endpoints doivent être versionnés (ex. /api/v1), documentés avec OpenAPI/Swagger, retourner des codes HTTP cohérents, paginer les listes importantes et filtrer côté serveur.

9. Exigences UI/UX

- Interface professionnelle, rapide, claire et adaptée au contexte d'un restaurant.
- Actions fréquentes accessibles en peu de clics/touches.
- Boutons de commande suffisamment grands sur tablette/mobile.
- États de commande visuellement distincts, sans dépendre uniquement de la couleur.
- Messages d'erreur en français, compréhensibles et actionnables.
- Confirmation avant annulation ou opération destructive.
- Écrans avec chargement, état vide, erreur et succès.
- Responsive pour le web ; Android/tablette prioritaire pour React Native.
- Identité visuelle LYNE RESTAURANT appliquée sans surcharger l'interface.

10. Sécurité, intégrité et sauvegarde

- HTTPS obligatoire en production.
- Mots de passe hachés ; jamais stockés en clair.
- Secrets et identifiants DB dans variables d'environnement, jamais dans Git.
- RBAC vérifié côté API sur chaque action protégée.
- Requêtes paramétrées/ORM ; protection contre injections SQL.
- Validation et sanitation des entrées ; limites de taille et type pour uploads.

- Rate limiting sur authentification et endpoints sensibles.
- Journal d'audit protégé contre modification par les utilisateurs standards.
- Sauvegarde automatique MySQL au minimum quotidienne avec politique de rétention.
- Procédure de restauration testée avant mise en production.
- Pas de données réelles sensibles dans les environnements de test.

11. Stratégie de tests et définition de terminé

- Tests unitaires pour calculs, permissions et règles métier critiques.
- Tests API d'intégration sur authentification, commandes, paiements, achats et stock.
- Tests de non-régression à chaque phase.
- Tests manuels des parcours principaux sur appareil réel.
- Test d'impression sur le modèle réel de mini-imprimante avant validation finale.
- Une phase n'est « terminée » que si : build réussi, migrations exécutables, tests passés, critères d'acceptation satisfaits, aucun bug bloquant, documentation mise à jour et validation utilisateur obtenue.

12. Plan de développement par phases

Phase	Objectif	Livrable / porte de sortie
Phase 0 — Cadrage & fondations	Valider identité, règles, stack, dépôt, environnements, conventions, maquettes essentielles.	Projet initialisé, README, .env.example, architecture, CI/build, charte UI.
Phase 1 — Base de données & API socle	Créer schéma MySQL, migrations, API, authentification, rôles/permissions, audit de base.	Login fonctionnel, seed admin, RBAC testé, /health OK.
Phase 2 — Personnel & administration	Employés, fonctions, services, comptes, cartes de service.	CRUD personnel complet, permissions, carte imprimable.
Phase 3 — Menu / Produits	Catégories, plats, prix, disponibilité, photos.	Catalogue exploitable par réception.
Phase 4 — Commandes	Création commande, lignes, tables/comptoir, états et historique.	Flux Nouvelle→Préparation→Prête→Service testé.
Phase 5 — Caisse, paiements & impression	Encaissement, clôture, ticket, mini-imprimante, réimpression.	Une commande payée produit un ticket sans doublon financier.
Phase 6 — Stocks	Articles, unités, mouvements, seuils, inventaire, alertes.	Stock traçable et alertes fonctionnelles.
Phase 7 — Achats & fournisseurs	Fournisseurs, facture achat, réception, justificatifs, dépenses.	Validation achat met à jour stock et finance une seule fois.
Phase 8 — Finances	Recettes, dépenses, caisse, synthèses par période.	Chiffres cohérents avec ventes/paiements/achats.
Phase 9 — Dashboard & rapports	KPI, graphiques, filtres, PDF/Excel.	Rapports vérifiés contre données source.
Phase 10 — Notifications & audit renforcé	Centre de notifications, logs complets, alertes.	Actions sensibles consultables et notifications pertinentes.
Phase 11 — QA, sécurité & optimisation	Tests E2E, permissions, performances, sauvegarde/restauration, corrections.	Zéro bug bloquant ; restauration testée ; checklist sécurité validée.
Phase 12 — Production & formation	Déploiement, domaine, SSL, configuration restaurant, imprimante, formation.	Production accessible, comptes créés, sauvegardes actives, PV de recette.

13. Protocole obligatoire pour chaque agent IA / développeur

17. Lire le présent cahier des charges et l'état du dépôt avant toute modification.
18. Travailler uniquement sur la phase explicitement autorisée.
19. Faire un diagnostic des fichiers concernés avant de coder.
20. Présenter un mini-plan d'exécution de la phase ; ne pas inventer de module hors périmètre.
21. Créer les migrations de manière non destructive et reproductible.
22. Implémenter frontend/mobile + API + DB nécessaires à la phase, sans casser les phases validées.
23. Exécuter lint, tests et build.

24. Tester avec des comptes et données de démonstration.
25. Fournir un rapport : réalisé, fichiers modifiés, DB/migrations, endpoints, tests, résultats, points restant.
26. Attendre la validation avant de démarrer la phase suivante.
27. Après validation, créer un commit clair et mettre à jour le journal de projet/AGENTS.md.

14. Critères de recette globale

Domaine	Critère d'acceptation
Authentification	Chaque rôle accède uniquement à ses fonctions.
Commande	Une commande complète peut être créée, suivie, servie, payée et retrouvée.
Calcul	Les totaux sont corrects et recalculés côté serveur.
Ticket	Le ticket LYNE s'imprime correctement sur l'imprimante retenue.
Stock	Entrées/sorties et inventaires donnent un stock cohérent et traçable.
Achat	Une réception d'achat met à jour stock et dépense sans doublon.
Finance	Ventes, paiements et dépenses correspondent aux opérations enregistrées.
Personnel	Employé et carte de service peuvent être créés.
Rapports	Filtres par période et exports essentiels fonctionnent.
Audit	Les actions sensibles indiquent auteur, date et opération.
Sécurité	Permissions serveur, mots de passe, HTTPS, validation et sauvegarde sont opérationnels.
Stabilité	Aucun bug bloquant sur les parcours principaux.

15. Données initiales à demander à LYNE RESTAURANT

- Logo officiel en bonne qualité.
- Nom commercial exact, adresse et contacts à afficher.
- Liste des employés, fonctions et photos pour cartes.
- Liste des catégories et plats/boissons avec prix.
- Liste des articles de stock, unités et seuils souhaités.
- Liste initiale des fournisseurs.
- Modes de paiement utilisés.
- Texte de pied de ticket.
- Modèle exact de mini-imprimante et mode de connexion.
- Règles internes d'annulation, remise éventuelle et clôture de caisse.

16. Évolutions possibles après V1

- Recettes techniques : consommation automatique d'ingrédients selon les plats vendus.
- Multi-branches / plusieurs restaurants.
- Réservations de tables.
- Programme fidélité.
- Commande client en ligne et livraison.
- Intégration Mobile Money.
- Gestion RH avancée : présence, planning, paie.
- Mode offline avancé avec synchronisation.
- Écran cuisine/KDS dédié et imprimante cuisine.
- Analyse avancée des coûts, marges et rentabilité par plat.

17. Livrables du projet

- Code source complet et dépôt Git.
- Application React Native/Expo et build Android selon périmètre.
- Dashboard web React si retenu dans la mise en œuvre.
- API documentée.
- Schéma MySQL, migrations et données de seed.
- Documentation d'installation et déploiement.
- Documentation utilisateur essentielle.
- Modèle de ticket thermique.
- Rapport de tests et recette.
- Procédure de sauvegarde/restauration.
- Journal des phases et décisions techniques.

18. Validation

Ce document devient la référence fonctionnelle et technique après validation. Toute modification importante de périmètre doit être documentée, estimée et approuvée avant développement.

Pour LYNE RESTAURANT	Pour ALIKA-KONNECT
Nom : _____	Nom : _____
Date : _____	Date : _____
Signature : _____	Signature : _____

FIN DU CAHIER DES CHARGES — VERSION 1.0